Métodos HTTP: GET, POST, PUT, PATCH, DELETE, HEAD y OPTIONS

Una URL identifica **dónde** dirigimos la petición; el método indica **qué solicitamos** a ese recurso. `GET /productos/42` y `DELETE /productos/42` se dirigen a la misma URI, pero no piden la misma operación. El servidor define qué recursos y métodos admite, respetando el significado de cada método.

Dos propiedades que conviene distinguir

Seguro significa que el cliente no solicita modificar el estado del recurso. Un servidor puede registrar la visita o contar accesos al atender un GET: eso no convierte GET en un método para pedir cambios. No hay que crear una URL como `/eliminar/42` que borre datos cuando recibe GET: un navegador, una caché o un robot podrían visitarla sin esperar esa consecuencia.

Idempotente significa que repetir la **misma petición** tiene el mismo efecto previsto sobre el recurso que enviarla una vez. No significa que todas las respuestas sean idénticas. Por ejemplo, el primer DELETE puede devolver `204 No Content` y el segundo `404 Not Found`: después de ambos intentos el recurso sigue eliminado. Tampoco significa que no haya registros o costes en el servidor.

Método	Seguro	Idempotente	Intención habitual
GET	Sí	Sí	Solicitar una representación
HEAD	Sí	Sí	Pedir los campos que se obtendrían con GET, sin cuerpo de respuesta
POST	No necesariamente	No necesariamente	Solicitar que el recurso procese el contenido
PUT	No	Sí	Crear o sustituir el estado del recurso objetivo
PATCH	No	No necesariamente	Aplicar una modificación parcial
DELETE	No	Sí	Solicitar que se elimine el recurso objetivo
OPTIONS	Sí	Sí	Consultar opciones de comunicación disponibles

No necesariamente no significa «siempre que se use POST habrá un cambio»: la semántica de POST depende del recurso. PATCH puede diseñarse para ser idempotente, pero el método por sí solo no lo garantiza. Las propiedades describen el contrato que el cliente debe poder esperar, no una tabla de códigos de respuesta que deban aparecer siempre.

Pedir información: GET, HEAD y OPTIONS

GET pide una representación del recurso. Por ejemplo:

```
GET /productos/42 HTTP/1.1
Host: tienda.example
Accept: application/json
```

El servidor podría responder `200 OK` con JSON; también podría responder `404 Not Found` si no ofrece ese producto. El método **no garantiza** ni el éxito ni que el contenido sea JSON: `Accept` expresa una preferencia del cliente y `Content-Type` describe lo que finalmente se envía.

HEAD solicita una respuesta con los campos que se obtendrían mediante GET, **sin cuerpo**. Puede ser útil para observar un `Content-Type` o la fecha de modificación sin descargar el contenido. No todas las aplicaciones ofrecen HEAD para todas sus rutas.

OPTIONS pregunta por las opciones de comunicación de un recurso. Si el servidor indica `Allow`, esa cabecera informa de métodos admitidos para ese recurso; **no hay que presuponer que todas las respuestas OPTIONS la incluyen**. Tampoco se debe deducir la capacidad de toda la aplicación a partir de un solo endpoint.

Solicitar un procesamiento: POST

POST pide al recurso que procese la representación que enviamos. Es común al crear un pedido, pero también puede representar enviar un formulario, solicitar un cálculo o iniciar un trabajo. «POST = crear» se queda corto: el resultado depende de lo que implemente `/pedidos` o `/consultas`.

```
POST /pedidos HTTP/1.1
Host: tienda.example
Content-Type: application/json

{"productoId":42,"cantidad":1}
```

Un servidor que cree el pedido podría responder `201 Created` y una cabecera `Location` con la URI nueva. Es solo un **ejemplo hipotético**, no un endpoint de Hello Server. Repetir una petición de compra podría crear otro pedido si la aplicación no define cómo evitarlo: no supongas idempotencia en POST.

Cambiar un recurso: PUT, PATCH y DELETE

PUT solicita crear o **reemplazar** el estado del recurso identificado por la URI. Si se envía dos veces la misma representación a `PUT /productos/42`, el estado previsto tras la segunda petición sigue siendo el mismo que tras la primera. Esto no obliga a devolver el mismo estado HTTP en ambas: una creación inicial podría dar `201` y el reemplazo posterior `200` o `204`, según la implementación.

PATCH aplica un cambio parcial. El cuerpo y su `Content-Type` determinan cómo se expresa. Por ejemplo, un contrato de API podría interpretar `{"precio":20}` como «fijar el precio en 20»; repetirlo sería idempotente. Otro contrato podría pedir «incrementar el precio en 1», que sí acumularía efectos. No supongas que cualquier PATCH es idempotente ni que ese cuerpo JSON tenga el mismo significado en todas las APIs.

DELETE pide eliminar el recurso identificado. Repetirlo no debería pedir una segunda eliminación distinta, pero la respuesta puede variar porque el recurso ya no exista. Idempotencia habla del **efecto solicitado**, no de obtener siempre `204`.

Qué podemos observar con Hello Server

Hello Server sirve aquí para mirar HTTP, **no** implementa el catálogo y los pedidos hipotéticos anteriores. Desde otra terminal, con la aplicación arrancada:

```
http --verbose GET http://localhost:8080/api/hello
http --verbose HEAD http://localhost:8080/api/hello
http --verbose OPTIONS http://localhost:8080/api/hello
```

1. ¿En cuál de las respuestas se ve un cuerpo? ¿Qué cambia con HEAD?
2. ¿Qué estados y cabeceras aparecen **realmente** al usar OPTIONS en este entorno? No presupongas `Allow` ni que todos los servidores actúan igual.
3. ¿Por qué no tendría sentido enviar `PUT /api/hello` para «cambiar» el saludo sin haber programado antes ese comportamiento?

Como comprobación final, clasifica un GET que lee datos, un POST que intenta crear un pedido y un DELETE repetido en términos de **intención, seguridad e idempotencia**. Después pasa al documento 07: estados HTTP para interpretar las respuestas observadas. Diseñar un CRUD o una API REST completa corresponde a UD02.